Vault Guardians Audit Report

Version 1.0

Cemerian

March 9, 2026

Vault Guardians Audit Report

Cemerian

March 9, 2026

Prepared by: Cemerian Lead Auditors: - Cemerian

Table of Contents

- Table of Contents
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
- Protocol Summary
  - Roles
- Executive Summary
  - Issues Found
- Findings
  - High
    * [H-1] Lack of UniswapV2 slippage protection in `UniswapAdapter` enables frontrunners to steal profits
    * [H-2] `rebalanceFunds()` is publicly callable, allowing forced divest/invest cycles that drain vault value
    * [H-3] Guardians can infinitely mint `VaultGuardianToken`s and take over DAO
    * [H-4] `ERC4626::totalAssets` only checks vault-held balance, ignoring invested assets, resulting in incorrect share calculations

- Medium
  * [M-1] Incorrect vault name and symbol for Token Two (LINK) vaults
  * [M-2] `becomeGuardian()` does not enforce the `GUARDIAN_FEE`
- Low
  * [L-1] `withdraw()` and `redeem()` trigger full divest/reinvest cycle, risking gas limit DoS
  * [L-2] `updateGuardianStakePrice()` emits incorrect event values, and `updateGuardianAndDaoCut()` emits the wrong event

Disclaimer

The Cemerian team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
357d766
```

Scope

```
./src/
#-- abstract
|   #-- AStaticTokenData.sol
|   #-- AStaticUSDCData.sol
|   #-- AStaticWethData.sol
#-- dao
|   #-- VaultGuardianGovernor.sol
|   #-- VaultGuardianToken.sol
#-- interfaces
|   #-- IVaultData.sol
|   #-- IVaultGuardians.sol
|   #-- IVaultShares.sol
|   #-- InvestableUniverseAdapter.sol
#-- protocol
|   #-- VaultGuardians.sol
|   #-- VaultGuardiansBase.sol
|   #-- VaultShares.sol
|   #-- investableUniverseAdapters
|       #-- AaveAdapter.sol
|       #-- UniswapAdapter.sol
#-- vendor
    #-- DataTypes.sol
    #-- IPool.sol
    #-- IUniswapV2Factory.sol
    #-- IUniswapV2Router01.sol
```

Protocol Summary

This protocol allows users to deposit certain ERC20s into an ERC4626 vault managed by a human being, or a `vaultGuardian`. The goal of a `vaultGuardian` is to manage the vault in a way that maximizes the value of the vault for the users who have deposited money into the vault.

Vault guardians are only allowed to deposit and withdraw ERC20s into specific protocols (the "investable universe"):

- Aave v3
- Uniswap v2
- None (just hold)

The guardian can move funds in and out of these protocols as much as they like, but they cannot move funds to any other address. Vault guardians charge a performance fee — the better the guardians do, the larger fee they will earn.

Roles

There are 4 main roles associated with the system.

- *Vault Guardian DAO*: The org that takes a cut of all profits, controlled by the `VaultGuardianToken`. The DAO controls a few variables of the protocol, including:
  - `s_guardianStakePrice`
  - `s_guardianAndDaoCut`
  - And takes a cut of the ERC20s made from the protocol
- *DAO Participants*: Holders of the `VaultGuardianToken` who vote and take profits on the protocol
- *Vault Guardians*: Strategists/hedge fund managers who have the ability to move assets in and out of the investable universe. They take a cut of revenue from the protocol.
- *Investors*: The users of the protocol. They deposit assets to gain yield from the investments of the Vault Guardians.

Executive Summary

The Vault Guardians project takes novel approaches to integrate ERC-4626 into a managed fund system, but contains several critical vulnerabilities around slippage protection, governance token economics, and balance tracking. The most severe issues allow MEV extraction, governance takeover, and share dilution through incorrect asset accounting.

Issues Found

Severity	Number of issues found
High	4
Medium	2
Low	2
Info	0
Gas	0
Total	8

Findings

High

[H-1] Lack of UniswapV2 slippage protection in `UniswapAdapter` enables frontrunners to steal profits

Description: In `UniswapAdapter.sol`, all four external calls to the Uniswap router use `amountOutMin: 0` or `amountAMin: 0` / `amountBMin: 0`. This means the contract will accept any amount of output tokens, no matter how unfavorable the price.

- `_uniswapInvest` → `swapExactTokensForTokens` at L54-61 with `amountOutMin: 0`
- `_uniswapInvest` → `addLiquidity` at L73-84 with `amountAMin: 0`, `amountBMin: 0`
- `_uniswapDivest` → `removeLiquidity` at L96-105 with `amountAMin: 0`, `amountBMin: 0`
- `_uniswapDivest` → `swapExactTokensForTokens` at L107-114 with `amountOutMin: 0`

Additionally, `deadline: block.timestamp` is used everywhere, which provides no deadline protection since `block.timestamp` is set by the miner/validator at execution time. In the PoS model, proposers know well in advance if they will propose consecutive blocks, allowing a malicious validator to hold the transaction and execute it at a more favorable block.

Impact: An attacker can sandwich every `deposit()`, `withdraw()`, `redeem()`, and `rebalanceFunds()` call. They front-run the vault's swap with a large buy to move the price, the vault swaps at the inflated price, then the attacker back-runs to sell at profit. This can extract significant value from the vault on every single rebalance or user interaction. Given that `rebalanceFunds()` is callable by anyone (see H-2), an attacker can trigger this extraction at will.

Proof of Concept:

1. User calls `VaultShares::deposit` with a vault that has a Uniswap allocation. This calls `_uniswapInvest`, which calls the router's `swapExactTokensForTokens`.
2. In the mempool, a malicious user:
    1. Holds onto the transaction which makes the Uniswap swap
    2. Takes a flashloan out
    3. Makes a major swap on Uniswap, greatly changing the price of the assets
    4. Executes the held transaction, giving the protocol as little funds back as possible due to `amountOutMin` being set to 0

3. The attacker back-runs, reversing their position at profit. The vault (and its depositors) absorb the loss.

Recommended Mitigation:

For the deadline issue:

DeFi is a large landscape. For protocols that have sensitive investing parameters, add a custom parameter to the `deposit` function so the Vault Guardians protocol can account for the customizations of DeFi projects that it integrates with. Consider allowing for custom data:

```
- function deposit(uint256 assets, address receiver) public override(ERC4626, IERC4626) isActive returns (uint256) {
+ function deposit(uint256 assets, address receiver, bytes customData) public override(ERC4626, IERC4626) isActive returns (uint256) {
```

This way, you could add a `deadline` to the Uniswap swap, and also allow for more DeFi custom integrations.

For the `amountOutMin` issue:

1. Do a price check on something like a Chainlink price feed before making the swap, reverting if the rate is too unfavorable.
2. Only deposit 1 side of a Uniswap pool for liquidity. Don't make the swap at all. If a pool doesn't exist or has too low liquidity for a pair of ERC20s, don't allow investment in that pool.

Note that these recommendations require significant changes to the codebase.

[H-2] `rebalanceFunds()` is publicly callable, allowing forced divest/invest cycles that drain vault value

Description: In `VaultShares.sol` L183, `rebalanceFunds()` is a **`public`** function with no access restriction. It triggers a full divest from Uniswap and Aave, then reinvests everything according to the current allocation.

```
function rebalanceFunds() public isActive divestThenInvest nonReentrant {}
```

Impact: Combined with the zero-slippage bug (H-1), any external actor (especially a Uniswap LP or MEV bot) can call `rebalanceFunds()` repeatedly. Each call divests all Uniswap liquidity (remove liquidity + swap counterparty token back) then reinvests (swap half of asset + add liquidity). Every swap is sandwichable with `amountOutMin: 0`. A Uniswap LP earns swap fees on each forced trade. An MEV bot profits from sandwich attacks. The vault loses value each cycle.

Proof of Concept:

1. A Uniswap LP or MEV bot calls `rebalanceFunds()` on an active vault.
2. The vault divests: removes liquidity and swaps the counterparty token back to the vault asset (2 transactions with 0 slippage protection).
3. The vault reinvests: swaps half the asset to the counterparty token and adds liquidity (2 more transactions with 0 slippage protection).
4. The attacker repeats, extracting value each time through swap fees or sandwich profits.

Recommended Mitigation: Restrict `rebalanceFunds()` to the guardian only:

```
- function rebalanceFunds() public isActive divestThenInvest
    nonReentrant {}
+ function rebalanceFunds() public onlyGuardian isActive
    divestThenInvest nonReentrant {}
```

[H-3] Guardians can infinitely mint `VaultGuardianTokens` and take over DAO

Description: Becoming a guardian comes with the perk of getting minted Vault Guardian Tokens (vg-Tokens). Whenever a guardian successfully calls `VaultGuardiansBase::becomeGuardian` or `VaultGuardiansBase::becomeTokenGuardian`, `_becomeTokenGuardian` is executed, which mints the caller `i_vgToken`:

```
function _becomeTokenGuardian(IERC20 token, VaultShares tokenVault)
    private returns (address) {
     s_guardians[msg.sender][token] = IVaultShares(address(tokenVault));
     emit GuardianAdded(msg.sender, token);
     i_vgToken.mint(msg.sender, s_guardianStakePrice);
     token.safeTransferFrom(msg.sender, address(this),
        s_guardianStakePrice);
     // ...
}
```

Guardians are also free to quit their role at any time via `VaultGuardianBase::quitGuardian`. However, `_quitGuardian()` at L242 never burns the VGT tokens. The combination of minting vgTokens on entry and never burning them on exit results in users being able to farm vgTokens at will.

Furthermore, a guardian can also simply withdraw their vault shares after becoming a guardian (noted at L283), keeping VGT while no longer having skin in the game.

Impact: Assuming the token has no monetary value, the malicious guardian could accumulate tokens until they can overtake the DAO. Since `VaultGuardianGovernor` uses VGT for voting with only a 4% quorum, an attacker can eventually pass arbitrary governance proposals. They could then execute any of these functions on the `VaultGuardians` contract:

- `sweepErc20s(address)` — drain all ERC20 balances
- `transferOwnership(address)` — take over ownership
- `updateGuardianAndDaoCut(uint256)` — manipulate fee structure
- `updateGuardianStakePrice(uint256)` — set stake price to 0

Proof of Concept:

1. Attacker calls `becomeGuardian()` with 10 WETH stake. Receives 10 VGT.
2. Attacker calls `quitGuardian()`. Gets stake back. Still holds 10 VGT.
3. Attacker repeats N times. Now holds N * 10 VGT.
4. Attacker delegates VGT to themselves and proposes a governance action.
5. With enough VGT to meet 4% quorum, attacker passes and executes the proposal.

Proof of Code

Place the following code into `VaultGuardiansBaseTest.t.sol`:

```
function testDaoTakeover() public hasGuardian hasTokenGuardian {
    address maliciousGuardian = makeAddr("maliciousGuardian");
    uint256 startingVoterUsdcBalance = usdc.balanceOf(maliciousGuardian);
    uint256 startingVoterWethBalance = weth.balanceOf(maliciousGuardian);
    assertEq(startingVoterUsdcBalance, 0);
    assertEq(startingVoterWethBalance, 0);

    VaultGuardianGovernor governor = VaultGuardianGovernor(payable(vaultGuardians.owner()));
    VaultGuardianToken vgToken = VaultGuardianToken(address(governor.token()));

    // Flash loan the tokens, or just buy a bunch for 1 block
    weth.mint(mintAmount, maliciousGuardian);
    uint256 startingMaliciousVGTokenBalance = vgToken.balanceOf(maliciousGuardian);
    uint256 startingRegularVGTokenBalance = vgToken.balanceOf(guardian);
    console.log("Malicious vgToken Balance:\t", startingMaliciousVGTokenBalance);
    console.log("Regular vgToken Balance:\t", startingRegularVGTokenBalance);

    // Malicious Guardian farms tokens
    vm.startPrank(maliciousGuardian);
    weth.approve(address(vaultGuardians), type(uint256).max);
    for (uint256 i; i < 10; i++) {
        address maliciousWethSharesVault = vaultGuardians.becomeGuardian(allocationData);
        IERC20(maliciousWethSharesVault).approve(
```

```
            address(vaultGuardians),
            IERC20(maliciousWethSharesVault).balanceOf(maliciousGuardian)
        );
        vaultGuardians.quitGuardian();
    }
    vm.stopPrank();

    uint256 endingMaliciousVGTokenBalance = vgToken.balanceOf(maliciousGuardian);
    uint256 endingRegularVGTokenBalance = vgToken.balanceOf(guardian);
    console.log("Malicious vgToken Balance:\t", endingMaliciousVGTokenBalance);
    console.log("Regular vgToken Balance:\t", endingRegularVGTokenBalance);
}
```

Recommended Mitigation: There are a few options to fix this issue:

1. Burn vgTokens when a guardian quits:

```
function _quitGuardian(IERC20 token) private returns (uint256) {
    IVaultShares tokenVault = IVaultShares(s_guardians[msg.sender][token]);
    s_guardians[msg.sender][token] = IVaultShares(address(0));
    emit GaurdianRemoved(msg.sender, token);
    tokenVault.setNotActive();
+   i_vgToken.burn(msg.sender, s_guardianStakePrice);
    uint256 maxRedeemable = tokenVault.maxRedeem(msg.sender);
    uint256 numberOfAssetsReturned = tokenVault.redeem(maxRedeemable, msg.sender, msg.sender);
    return numberOfAssetsReturned;
}
```

Also add a `burn` function to `VaultGuardianToken`.

2. Mint vgTokens on a vesting schedule after a user becomes a guardian.
3. Simply don't allocate vgTokens to guardians. Instead, mint the total supply on contract deployment.

[H-4] `ERC4626::totalAssets` only checks vault-held balance, ignoring invested assets, resulting in incorrect share calculations

Description: The `ERC4626::totalAssets` function checks the balance of the underlying asset for the vault using the `balanceOf` function:

```
function totalAssets() public view virtual returns (uint256) {
```

```
    return _asset.balanceOf(address(this));
}
```

However, the assets are invested in the investable universe (Aave and Uniswap) which means this will never return the correct value of assets in the vault. In `VaultShares.sol` L152, `previewDeposit(assets)` relies on this incorrect `totalAssets()` to calculate shares for new depositors.

Impact: This breaks many functions of the `ERC4626` contract: - `totalAssets` - `convertToShares` - `convertToAssets` - `previewWithdraw` - `withdraw` - `deposit`

If a vault has 10 WETH total value but only 2 WETH held (the rest in Aave/Uniswap), a new depositor putting in 2 WETH would get shares as if the vault only had 2 WETH total — minting far more shares than they deserve. This dilutes existing depositors' share of the vault. Conversely, when the vault is fully divested (during `withdraw`/`redeem` via `divestThenInvest`), `totalAssets` temporarily reflects the true total, creating an inconsistency that can be exploited.

Proof of Concept:

Proof of Code

Add the following code to the `VaultSharesTest.t.sol` file:

```
function testWrongBalance() public {
    // Mint 100 ETH
    weth.mint(mintAmount, guardian);
    vm.startPrank(guardian);
    weth.approve(address(vaultGuardians), mintAmount);
    address wethVault = vaultGuardians.becomeGuardian(allocationData);
    wethVaultShares = VaultShares(wethVault);
    vm.stopPrank();

    // prints 3.75 ETH (should be ~10 ETH)
    console.log(wethVaultShares.totalAssets());

    // Mint another 100 ETH
    weth.mint(mintAmount, user);
    vm.startPrank(user);
    weth.approve(address(wethVaultShares), mintAmount);
    wethVaultShares.deposit(mintAmount, user);
    vm.stopPrank();

    // prints 41.25 ETH (significantly wrong)
    console.log(wethVaultShares.totalAssets());
}
```

Recommended Mitigation: Do not use the OpenZeppelin implementation of the `ERC4626` contract's `totalAssets`. Instead, natively keep track of users' total amounts sent to each protocol.

Override `totalAssets()` to include Aave aToken balance and Uniswap LP token value:

```
function totalAssets() public view override returns (uint256) {
    uint256 held = IERC20(asset()).balanceOf(address(this));
    uint256 inAave = i_aaveAToken.balanceOf(address(this));
    uint256 inUniswap = _getUniswapPositionValue(); // calculate LP value in terms of asset
    return held + inAave + inUniswap;
}
```

Potentially have an automation tool or some incentivised mechanism to keep track of protocol's profits and losses, and take snapshots of the investable universe. This would take a considerable re-write of the protocol.

Medium

[M-1] Incorrect vault name and symbol for Token Two (LINK) vaults

Description: In `VaultGuardiansBase.sol` L174-176, when creating a vault for `i_tokenTwo` (LINK), the constructor uses `TOKEN_ONE_VAULT_NAME` and `TOKEN_ONE_VAULT_SYMBOL` instead of `TOKEN_TWO_VAULT_NAME` and `TOKEN_TWO_VAULT_SYMBOL`.

```
} else if (address(token) == address(i_tokenTwo)) {
    tokenVault =
    new VaultShares(IVaultShares.ConstructorData({
        asset: token,
        vaultName: TOKEN_ONE_VAULT_NAME,    // "Vault Guardian USDC" instead of "Vault Guardian LINK"
        vaultSymbol: TOKEN_ONE_VAULT_SYMBOL, // "vgUSDC" instead of "vgLINK"
        ...
    }));
```

The correct constants exist in `AStaticTokenData.sol`: `TOKEN_TWO_VAULT_NAME = "Vault Guardian LINK"` and `TOKEN_TWO_VAULT_SYMBOL = "vgLINK"`.

Impact: LINK vaults will have the ERC20 name "Vault Guardian USDC" and symbol "vgUSDC", misleading wallets, block explorers, and integrators. Users may confuse LINK vaults for USDC vaults, leading to incorrect investment decisions. Any protocol or aggregator integrating by vault name/symbol will misidentify these vaults.

Proof of Concept:

1. Guardian calls `becomeTokenGuardian(allocationData, linkToken)`.
2. The deployed `VaultShares` contract has `name()= "Vault Guardian USDC"` and `symbol()= "vgUSDC"`.

3. The vault actually holds LINK as its underlying asset.

Recommended Mitigation:

```
  } else if (address(token) == address(i_tokenTwo)) {
      tokenVault =
      new VaultShares(IVaultShares.ConstructorData({
          asset: token,
-         vaultName: TOKEN_ONE_VAULT_NAME,
-         vaultSymbol: TOKEN_ONE_VAULT_SYMBOL,
+         vaultName: TOKEN_TWO_VAULT_NAME,
+         vaultSymbol: TOKEN_TWO_VAULT_SYMBOL,
```

Also, add a new test in the `VaultGuardiansBaseTest.t.sol` file to avoid reintroducing this error, similar to what's done in the test `testBecomeTokenGuardianTokenOneName`.

[M-2] `becomeGuardian()` does not enforce the `GUARDIAN_FEE`

Description: In `VaultGuardiansBase.sol` L123-124, the `GUARDIAN_FEE` constant is defined at L65 as `0.1 ether`, but `becomeGuardian()` is not `payable` and never checks `msg.value`:

```
uint256 private constant GUARDIAN_FEE = 0.1 ether;
// ...
function becomeGuardian(AllocationData memory wethAllocationData)
    external returns (address) {
```

Impact: The protocol never collects the intended ETH fee from guardians. This is lost revenue for the DAO and removes a financial barrier to becoming a guardian, making governance attacks (H-3) cheaper.

Recommended Mitigation: Make the function `payable` and enforce the fee:

```
- function becomeGuardian(AllocationData memory wethAllocationData)
    external returns (address) {
+ function becomeGuardian(AllocationData memory wethAllocationData)
    external payable returns (address) {
+     if (msg.value < GUARDIAN_FEE) {
+         revert VaultGuardiansBase__FeeTooSmall(msg.value,
    GUARDIAN_FEE);
+     }
```

Low

[L-1] `withdraw()` and `redeem()` trigger full divest/reinvest cycle, risking gas limit DoS

Description: In `VaultShares.sol` L191-203, both `withdraw()` and `redeem()` use the `divestThenInvest` modifier, which removes all liquidity from Uniswap and Aave, then reinvests the remaining balance after the withdrawal. This involves multiple external calls (remove liquidity, swap, add liquidity, Aave withdraw, Aave supply).

Impact: For vaults with large positions or during periods of high gas costs, the total gas consumed may exceed the block gas limit, making withdrawals impossible. Users' funds would be temporarily locked until gas conditions change or the guardian changes allocation to reduce external protocol interactions.

Recommended Mitigation: Consider a proportional withdrawal approach that only divests the portion needed, rather than divesting and reinvesting the entire vault balance on every withdrawal.

[L-2] `updateGuardianStakePrice()` emits incorrect event values, and `updateGuardianAndDaoCut()` emits the wrong event

Description: In `VaultGuardians.sol` L71-74, `updateGuardianStakePrice` sets `s_guardianStakePrice = newStakePrice` before emitting the event. The event then logs `s_guardianStakePrice` as the "old" value — but it's already been overwritten:

```
function updateGuardianStakePrice(uint256 newStakePrice) external
    onlyOwner {
    s_guardianStakePrice = newStakePrice;
    emit VaultGuardians__UpdatedStakePrice(s_guardianStakePrice,
        newStakePrice);
    // Both args are now newStakePrice — old value is lost
}
```

Additionally, `updateGuardianAndDaoCut()` at L82-85 emits `VaultGuardians__UpdatedStakePrice` instead of `VaultGuardians__UpdatedFee`:

```
function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner {
    s_guardianAndDaoCut = newCut;
    emit VaultGuardians__UpdatedStakePrice(s_guardianAndDaoCut, newCut)
        ; // wrong event
}
```

Impact: Off-chain monitoring, indexing services, and governance dashboards will receive incorrect data. The old stake price is never emitted, and fee updates appear as stake price updates in event logs.

Recommended Mitigation:

```
function updateGuardianStakePrice(uint256 newStakePrice) external onlyOwner {
+   uint256 oldStakePrice = s_guardianStakePrice;
    s_guardianStakePrice = newStakePrice;
-   emit VaultGuardians__UpdatedStakePrice(s_guardianStakePrice, newStakePrice);
+   emit VaultGuardians__UpdatedStakePrice(oldStakePrice, newStakePrice);
}

function updateGuardianAndDaoCut(uint256 newCut) external onlyOwner {
+   uint256 oldCut = s_guardianAndDaoCut;
    s_guardianAndDaoCut = newCut;
-   emit VaultGuardians__UpdatedStakePrice(s_guardianAndDaoCut, newCut);
+   emit VaultGuardians__UpdatedFee(oldCut, newCut);
}
```